Name : Gaurang Tyagi
Roll No. : 16

Assignment - 7

Ques1. Use MATLAB interface commands such as clc, clear, who, whos, and help, and create variables including a scalar, vectors, and a complex number, performing type conversion (double, single, int32) and comparing precision.

CODE

```
clc
clear

% Scalar
a = 10;

% Row vector
v1 = [1 2 3 4 5];

% Column vector
v2 = [1;2;3;4;5];

% Complex number
z = 3 + 4i;

% Type conversion
a_double = double(a);
a_single = single(a);
a_int = int32(a);

% Precision comparison
x_double = double(1/3);
x_single = single(1/3);

disp('Double precision:')
disp(x_double)

disp('Single precision:')
disp(x_single)

% Workspace info
who
whos

% Help command
help sqrt
```

OUTPUT

```
Double precision:
    0.3333

Single precision:
    0.3333


Your variables are:

a          a_int      v1         x_double  z
a_double   a_single   v2         x_single

  Name          Size            Bytes  Class     Attributes

  a             1x1                 8  double
  a_double      1x1                 8  double
  a_int         1x1                 4  int32
  a_single      1x1                 4  single
  v1            1x5                40  double
  v2            5x1                40  double
  x_double      1x1                 8  double
  x_single      1x1                 4  single
  z             1x1                16  double    complex

 sqrt - Square root
    This MATLAB function returns the square root of each element of the
    array X.

    Syntax
      B = sqrt(X)
```

Ques 2. Construct a 5×6 matrix A with defined patterns (first row as multiples of 3, last row as squares, remaining rows as random integers), and reshape it into different valid dimensions while preserving the total number of elements.

CODE

```
first_row = 3*(1:6);
middle_rows = randi(10,3,6);
last_row = (1:6).^2;

mat = [first_row;middle_rows;last_row];
display(mat);

% dimensions
size(mat);
display(ans);
```

```
% number of elements

numel(mat);
display(ans);

% reshaping

reshape(mat,1,30);
display(ans);

reshape(mat,2,15);
display(ans);

reshape(mat,3,10);
display(ans);

reshape(mat,5,6);
display(ans);

reshape(mat,6,5);
display(ans);

reshape(mat,10,3);
display(ans);

reshape(mat,15,2);
display(ans);

reshape(mat,30,1);
display(ans);
```

OUTPUT

```
mat =

     3     6     9    12    15    18
     6     6     2     9     9     2
     2     1     1     7     1     6
     7     1     5     6    10     2
     1     4     9    16    25    36


ans =

     5     6


ans =

    30


ans =

  Columns 1 through 15

     3     6     2     7     1     6     6     1     1     4     9     2     1     5     9

  Columns 16 through 30

    12     9     7     6    16    15     9     1    10    25    18     2     6     2    36
```

```
ans =

     3     2     1     6     1     9     1     9     9     6    15     1    25     2     2
     6     7     6     1     4     2     5    12     7    16     9    10    18     6    36


ans =

     3     7     6     4     1    12     6     9    25     6
     6     1     1     9     5     9    16     1    18     2
     2     6     1     2     9     7    15    10     2    36


ans =

     3     6     9    12    15    18
     6     6     2     9     9     2
     2     1     1     7     1     6
     7     1     5     6    10     2
     1     4     9    16    25    36


ans =

     3     6     1     6    25
     6     1     5    16    18
     2     1     9    15     2
     7     4    12     9     6
     1     9     9     1     2
```

```
ans =

     3     9    15
     6     2     9
     2     1     1
     7     5    10
     1     9    25
     6    12    18
     6     9     2
     1     7     6
     1     6     2
     4    16    36


ans =

     3    12
     6     9
     2     7
     7     6
     1    16
     6    15
     6     9
     1     1
     1    10
     4    25
     9    18
     2     2
```

Ques 3. Apply advanced indexing and slicing techniques to extract submatrices, select specific rows/columns, and modify elements using logical conditions; also perform horizontal and vertical concatenation with another compatible matrix.

CODE

```
clc
clear

A = reshape(1:20,4,5);

% Submatrix
subA = A(2:3,2:4);

% Select rows
rows = A([1 3],:);

% Select columns
cols = A(:,[2 5]);

% Logical indexing
A(A>10) = 0;

% Even numbers modification
A(mod(A,2)==0) = -1;
```

```
% Horizontal concatenation
B = ones(4,2);
H = [A B];

% Vertical concatenation
C = zeros(2,5);
V = [A; C];

disp(A)
disp(H)
disp(V)
```

OUTPUT

```
     1     5     9    -1    -1
    -1    -1    -1    -1    -1
     3     7    -1    -1    -1
    -1    -1    -1    -1    -1

     1     5     9    -1    -1     1     1
    -1    -1    -1    -1    -1     1     1
     3     7    -1    -1    -1     1     1
    -1    -1    -1    -1    -1     1     1

     1     5     9    -1    -1
    -1    -1    -1    -1    -1
     3     7    -1    -1    -1
    -1    -1    -1    -1    -1
     0     0     0     0     0
     0     0     0     0     0

>> ✨ Press Ctrl + Shift + P to generate code with Copilot
```

Ques4. Extract a square submatrix from A and perform matrix operations including transpose, determinant, inverse (if it exists), and eigenvalue computation, and determine whether the matrix is singular or non-singular.

CODE

```
A = reshape(1:24, 6, 4)';
B = A(1:3, 1:3);
B_T = B';
detB = det(B);

if detB == 0
```

```
    disp('Matrix is Singular');
else
    disp('Matrix is Non-Singular');
end

if detB ~= 0
    B_inv = inv(B);
else
    disp('Inverse does not exist');
end

eigB = eig(B);

disp('Submatrix B:');
disp(B);

disp('Transpose:');
disp(B_T);

disp('Determinant:');
disp(detB);

disp('Inverse:');
disp(B_inv);

disp('Eigenvalues:');
disp(eigB);
```

OUTPUT

```
Matrix is Non-Singular
Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND
1.027984e-18.

Submatrix B:
     1     2     3
     7     8     9
    13    14    15

Transpose:
     1     7    13
     2     8    14
     3     9    15

Determinant:
  -1.3323e-15

Inverse:
   1.0e+16 *

    0.4504   -0.9007    0.4504
   -0.9007    1.8014   -0.9007
    0.4504   -0.9007    0.4504

Eigenvalues:
   25.4164
   -1.4164
    0.0000
```

Ques 5. Create a new 5×7 matrix with distinct row-wise patterns (arithmetic progression, geometric progression, Fibonacci sequence, prime numbers, alternating signs), reshape it if possible into a square matrix, compute eigenvalues, and compare them with those obtained earlier.

CODE

```
% Arithmetic progression
r1 = 2:2:14;

% Geometric progression
r2 = 2 * 3.^(0:6);

% Fibonacci
r3 = zeros(1,7);
r3(1:2) = [1 1];
for i=3:7
    r3(i) = r3(i-1)+r3(i-2);
end

% Prime numbers
r4 = primes(20);
r4 = r4(1:7);
```

```
% Alternating signs
r5 = (-1).^(0:6).*(1:7);

% Matrix
M = [r1;r2;r3;r4;r5];

disp(M)

% reshape attempt
if mod(sqrt(numel(M)),1)==0
    S = reshape(M,sqrt(numel(M)),sqrt(numel(M)));
else
    S = reshape(M(1:25),5,5);
end

% eigenvalues
eig_new = eig(S);
disp(eig_new)
```

OUTPUT

```
eig_new = eig(S);
disp(eig_new)
           2           4           6           8          10          12          14
           2           6          18          54         162         486        1458
           1           1           2           3           5           8          13
           2           3           5           7          11          13          17
           1          -2           3          -4           5          -6           7

   8.7414 +12.3133i
   8.7414 -12.3133i
   5.6412 + 0.0000i
  -0.5619 + 0.1416i
  -0.5619 - 0.1416i
```